

Pregunta 1. Preguntas Teoricas (6.0 puntos)

1. Consistencia vs. Admisibilidad

Conceptos Clave (Recordatorio):

- $h^*(n)$: Costo real del camino más barato desde el nodo n hasta el objetivo.
- **Admisibilidad:** $h(n) \leq h^*(n)$ para todo nodo n (nunca sobreestima el costo real).
- **Consistencia (Monotonía):** Para todo nodo n y su sucesor n' con costo de paso $c(n, n')$, se cumple que $h(n) \leq c(n, n') + h(n')$. Además, $h(Goal) = 0$.

1.1. Demostración: Consistencia $\implies$ Admisibilidad

Estrategia: Utilizar la desigualdad del triángulo de forma recursiva sobre un camino óptimo.

Pasos:

1. Sea $P = (n_0, n_1, n_2, \dots, n_k)$ un camino óptimo desde un nodo n (n_0) hasta el objetivo (n_k).
2. El costo real de este camino es $h^*(n) = \sum_{i=0}^{k-1} c(n_i, n_{i+1})$.
3. Aplicamos la propiedad de **consistencia** recursivamente a lo largo del camino:
 - $h(n_0) \leq c(n_0, n_1) + h(n_1)$
 - $h(n_1) \leq c(n_1, n_2) + h(n_2)$
 - ...
 - $h(n_{k-1}) \leq c(n_{k-1}, n_k) + h(n_k)$
4. Como n_k es el objetivo, por definición de consistencia $h(n_k) = 0$.
5. Sustituyendo hacia atrás (o sumando las desigualdades), obtenemos:
$$h(n) \leq c(n_0, n_1) + c(n_1, n_2) + \dots + c(n_{k-1}, n_k)$$
$$h(n) \leq h^*(n)$$
6. **Conclusión:** Dado que $h(n)$ nunca supera al costo real $h^*(n)$, la función heurística es **admisibile**.

1.2. Contraejemplo: Admisibilidad $\nRightarrow$ Consistencia

Para probar que la inversa no es cierta, necesitamos una heurística que no sobreestime el costo global, pero que viole la restricción local del triángulo.

Grafo de ejemplo:

- Nodos: $S \rightarrow A \rightarrow G$ (donde G es el objetivo).
- Costos de los arcos: $c(S, A) = 1$ y $c(A, G) = 1$.
- Por tanto, el costo real óptimo es: $h^*(S) = 2, h^*(A) = 1, h^*(G) = 0$.

Definición de la heurística $h(n)$:

- $h(S) = 2$
- $h(A) = 0$
- $h(G) = 0$

Verificación:

1. **¿Es admisible?**
 - $h(S) = 2 \leq h^*(S) = 2$ (Cumple)
 - $h(A) = 0 \leq h^*(A) = 1$ (Cumple)
 - $h(G) = 0 \leq h^*(G) = 0$ (Cumple)
 - $\rightarrow$ **Es admisible**.
2. **¿Es consistente?**
 - Verificamos la condición para el arco $S \rightarrow A$:
 - ¿Se cumple $h(S) \leq c(S, A) + h(A)$?
 - $2 \leq 1 + 0$
 - $2 \leq 1 \rightarrow$ **Falso**.
 - $\rightarrow$ **No es consistente**.

1.3. Implicaciones para Búsqueda en Tiempo Real (Real-Time Search)

La diferencia es crítica para algoritmos como LRTA* (Learning Real-Time A*) o agentes que deben moverse antes de terminar de calcular.

1. Reapertura de nodos (Graph Search):

- Si h es consistente, cuando un nodo es expandido (cerrado), se ha encontrado el camino óptimo hacia él. No es necesario "reabrirlo" ni actualizarlo posteriormente.
- Si h es solo admisible (inconsistente), al expandir un nodo vecino podemos encontrar un camino "más barato" hacia un nodo ya visitado. Esto obliga al algoritmo a reaperturar nodos y recalcular, lo cual es computacionalmente costoso.

2. Estabilidad en la toma de decisiones:

- La consistencia garantiza que los valores $f(n) = g(n) + h(n)$ son monótonos (no disminuyen) a lo largo de cualquier camino.
- En **búsqueda en tiempo real**, esto significa que el agente no se llevará sorpresas desagradables donde moverse a un nodo vecino haga que el costo estimado al objetivo baje drásticamente (efecto "valle"), lo que causaría que el agente oscile o retroceda constantemente. Una heurística consistente permite un aprendizaje y convergencia más suave y eficiente del agente.

2. Teorema de Optimalidad de A*

Demostración Formal:

1. **Hipótesis:** A* con heurística admisible $h(n) \leq C^*(n)$

2. **Notación:**

- C^* : costo del camino óptimo
- $g(n)$: costo actual hasta n
- $f(n) = g(n) + h(n)$

3. **Lema:** Para cualquier nodo n en camino óptimo:

$$f(n) = g(n) + h(n) \leq C^*$$

Demostración: $g(n)$ es costo real hasta n en camino óptimo, y $h(n) \leq C^*(n)$, entonces $g(n) + h(n) \leq g(n) + C^*(n) = C^*$

4. **Demostración por contradicción:**

- Suponer que A* retorna camino con costo $C > C^*$
- Sea G primer nodo objetivo expandido con $f(G) = g(G) = C$
- Antes de expandir G , A* expande nodos en orden creciente de f
- Existe nodo n en camino óptimo no expandido con $f(n) \leq C^* < C$
- Entonces A* debería expandir n antes que $G \rightarrow$ contradicción

5. **Conclusión:** A* retorna camino óptimo

Relación con Dominancia:

- Heurística h_2 domina h_1 si $h_2(n) \geq h_1(n) \forall n$
- Si ambas son admisibles, h_2 expande menos nodos (mejor f)
- A* con h_2 es más eficiente manteniendo optimalidad

3. Complejidad y Completeness en CSPs

Definiciones Formales:

- **Completeness:** Algoritmo A es completo para CSP si, dado cualquier CSP, A encuentra solución si existe, o reporta inconsistencia si no existe
- **Optimalidad:** Algoritmo A es óptimo si encuentra solución que minimiza función objetivo (si la hay)

Análisis de Complejidad:

1. **Backtracking simple:**

- Complejidad: $O(d^n)$ en peor caso
- Completo: sí
- Óptimo: solo si usa ordenamiento y poda por optimalidad

2. **Backtracking con Forward Checking (FC):**

- Complejidad: $O(d^n)$ pero con factor constante menor
- Reduce dominios al asignar variables
- Evita conflictos inmediatos
- Completo: sí

3. **Backtracking con AC-3:**

- Preprocesamiento: $O(ed^3)$ donde e = número de restricciones
- Mantiene consistencia de arco durante búsqueda
- Reduce espacio de búsqueda significativamente
- Completo: sí

Demostración Limitaciones AC-3:

Ejemplo: CSP con 3 variables {X, Y, Z}, dominios {1,2,3}, restricciones:

- Alldifferent(X, Y, Z)
- $X \neq 1, Y \neq 1, Z \neq 1$

AC-3 elimina valores inconsistentes por pares:

- $X \neq Y, X \neq Z, Y \neq Z$ (consistencia binaria)
- Pero no detecta que {2,3} para cada variable es inconsistente para Alldifferent

Contraejemplo formal: Variables: {A, B, C}, dominios: {1,2} Restricciones: $A \neq B, B \neq C, A \neq C$ AC-3:

- Para arco (A,B): elimina nada ($1 \neq 2, 2 \neq 1$)
- Para arco (B,C): igual
- Para arco (A,C): igual Resultado: dominios completos {1,2} Pero CSP es inconsistente: no hay asignación que satisfaga las tres restricciones

Conclusión: AC-3 garantiza consistencia de arco (2-consistencia) pero no garantiza consistencia global (k-consistencia para $k > 2$). Por tanto, no garantiza optimalidad ni eliminación completa de backtracking, pero reduce dramáticamente el espacio de búsqueda.

Pregunta 2. IA Basada en Datos y Reduccion de Dimensionalidad (4 puntos)

a) PCA: maximización de varianza proyectada y autovectores de la covarianza

Sea $X \in \mathbb{R}^{n \times m}$ con columnas centradas. La covarianza es $\Sigma = \frac{1}{n} X^\top X$. La proyección escalar de una muestra x sobre un vector unitario w es $z = w^\top x$. La varianza proyectada es

$$\text{Var}(z) = \mathbb{E}[(w^\top x)^2] = w^\top \mathbb{E}[xx^\top] w = w^\top \Sigma w.$$

El objetivo de PCA para el primer componente principal es

$$\max_{w \in \mathbb{R}^m} w^\top \Sigma w \quad \text{sujeto a} \quad w^\top w = 1.$$

Este es un cociente de Rayleigh. Usando multiplicadores de Lagrange, se define

$$\mathcal{L}(w, \lambda) = w^\top \Sigma w - \lambda(w^\top w - 1).$$

Al anular el gradiente respecto de (w), se obtiene

$$\nabla_w \mathcal{L} = 2\Sigma w - 2\lambda w = 0 \quad \Rightarrow \quad \Sigma w = \lambda w.$$

Por lo tanto, los óptimos son autovectores de Σ y los valores óptimos de la varianza son los autovalores correspondientes. Componentes posteriores se obtienen imponiendo ortogonalidad entre vectores w y seleccionando los siguientes autovectores ordenados por autovalor descendente.

b) LDA: matrices de dispersión y problema de autovalores generalizado

Sea el conjunto de clases $\{1, \dots, C\}$. Defina:

- **Media global:**

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i.$$

- **Media de clase (c) y tamaño (n_c):**

$$\mu_c = \frac{1}{n_c} \sum_{i: y_i=c} x_i, \quad n_c = |\{i : y_i = c\}|.$$

- **Dispersión intra-clase:**

$$S_W = \sum_{c=1}^C \sum_{i: y_i=c} (x_i - \mu_c)(x_i - \mu_c)^\top.$$

- **Dispersión inter-clase:**

$$S_B = \sum_{c=1}^C n_c (\mu_c - \mu)(\mu_c - \mu)^\top.$$

El criterio de Fisher para un vector w es

$$J(w) = \frac{w^\top S_B w}{w^\top S_W w}.$$

Maximizar $J(w)$ equivale a

$$\max_w w^\top S_B w \quad \text{sujeto a} \quad w^\top S_W w = 1.$$

Usando Lagrange,

$$\mathcal{L}(w, \lambda) = w^\top S_B w - \lambda(w^\top S_W w - 1).$$

Anulando el gradiente en w ,

$$\nabla_w \mathcal{L} = 2S_B w - 2\lambda S_W w = 0 \quad \Rightarrow \quad S_B w = \lambda S_W w.$$

Esto define un problema de autovalores generalizado. En el caso multiclase, las direcciones discriminantes se obtienen tomando los

autovectores asociados a los mayores autovalores (hasta $\min(m, C - 1)$), y la proyección W se forma apilando dichos autovectores como columnas.

c) Comparación crítica: PCA vs. LDA

- Objetivo de optimización:
 - PCA:** Maximiza varianza proyectada $w^\top \Sigma w$ sin considerar etiquetas; busca mejor reconstrucción/energía en el subespacio.
 - LDA:** Maximiza separabilidad entre clases vía $J(w)$, maximizando dispersión inter-clase relativa a la intra-clase.
- Uso de información de clases:
 - PCA:** No usa etiquetas; completamente no supervisado.
 - LDA:** Supervisión explícita; depende de y para definir S_W y S_B .
- Supuestos sobre la distribución:
 - PCA:** Eficaz bajo estructuras lineales y cuando la variabilidad relevante se alinea con direcciones de mayor varianza; no requiere modelo probabilístico explícito.
 - LDA:** Óptimo bajo clases gaussianas con covarianzas iguales (homocedásticas); bajo estos supuestos produce fronteras lineales Bayes-óptimas.
- Adecuación a tareas:
 - Clasificación:** LDA es preferible porque maximiza separabilidad; produce subespacios discriminantes.
 - Reducción no supervisada:** PCA es preferible cuando no hay etiquetas o se busca compresión/reconstrucción; útil como preprocesamiento.
 - Dimensionalidad máxima:** PCA puede extraer hasta m componentes (limitado por rango de Σ); LDA extrae como máximo $C - 1$ direcciones discriminantes.

Pregunta 3. Analisis Teorico - El Problema de la Separabilidad Lineal (3 pts)

Separabilidad lineal en clasificación binaria

- Definición formal:** Un conjunto de datos binario $\{(x_i, y_i)\}_{i=1}^n$ con $x_i \in \mathbb{R}^2$ y $y_i \in \{-1, +1\}$ es linealmente separable si existe $(w, b) \in \mathbb{R}^2 \times \mathbb{R}$ tal que

$$y_i (w^\top x_i + b) > 0 \quad \text{para todo } i.$$

Equivalentemente, hay una recta $\{x \in \mathbb{R}^2 : w^\top x + b = 0\}$ que coloca todos los puntos de clase $+1$ en un semiplano y los de clase -1 en el otro.

Configuración de XOR en $\mathbb{R}^2$

- Datos:**
Negativos: $(0, 0), (1, 1)$.
Positivos: $(0, 1), (1, 0)$.
Esta disposición ocupa los vértices de un cuadrado unidad, con clases asignadas en diagonales opuestas.
- Prueba por contradicción mediante desigualdades:** Sea $f(x) = w_1 x_1 + w_2 x_2 + b$. Exigir la separación para XOR implica:

$f(0, 0) = b < 0$

(negativo)

$f(1, 1) = w_1 + w_2 + b < 0$

(negativo)

$f(1, 0) = w_1 + b > 0$

(positivo)

$f(0, 1) = w_2 + b > 0$

(positivo)

Sumando las dos últimas:

$$(w_1 + b) + (w_2 + b) = w_1 + w_2 + 2b > 0.$$

Pero de $w_1 + w_2 + b < 0$ se deduce $w_1 + w_2 + 2b < 0$ al sumar $b < 0$, lo que contradice la desigualdad anterior. Por tanto, no existen w, b que separen linealmente XOR.
- Argumento geométrico (convexidad):** El conjunto positivo debe quedar dentro de un semiplano (convexo). Si $(0, 1)$ y $(1, 0)$ pertenecen al mismo semiplano, entonces todo el segmento entre ellos (incluido su punto medio $(\frac{1}{2}, \frac{1}{2})$) también pertenecería al semiplano positivo. Cualquier recta que intente colocar simultáneamente $(0, 0)$ y $(1, 1)$ en el semiplano opuesto forzosamente arrastra uno de los positivos al lado negativo, o viceversa. Esto muestra que no existe una frontera lineal consistente.

Pregunta 4. Autoencoders vs PCA: Fundamentos Teoricos (4 puntos)

4.1. Estructura Matemática (1.5 puntos)

Definición Formal: Un autoencoder es un modelo de aprendizaje no supervisado compuesto por dos funciones parametrizadas:

1. **Codificador (f):** Mapea los datos de entrada $\mathbf{x} \in \mathbb{R}^m$ a un espacio latente de menor dimensión (cuello de botella) $\mathbf{z} \in \mathbb{R}^d$.

$$f: \mathbb{R}^m \rightarrow \mathbb{R}^d, \quad \mathbf{z} = f(\mathbf{x})$$

Donde la dimensión del cuello de botella satisface $d < m$ para forzar la compresión de datos.

2. **Decodificador (g):** Reconstruye los datos originales a partir de la representación latente $\mathbf{z}$.

$$g: \mathbb{R}^d \rightarrow \mathbb{R}^m, \quad \hat{\mathbf{x}} = g(\mathbf{z}) = g(f(\mathbf{x}))$$

Objetivo de Optimización: El modelo se entrena encontrando los parámetros de f y g que minimizan la función de pérdida de reconstrucción (error cuadrático medio):

$$\min_{f,g} \mathcal{L} = \|\mathbf{x} - g(f(\mathbf{x}))\|^2$$

Esto equivale a minimizar la información perdida al comprimir los datos.

4.2. Equivalencia Lineal (1.5 puntos)

Objetivo: Demostrar que un Autoencoder lineal $\approx$ PCA.

Demostración:

1. **Linealidad:** Supongamos que no hay funciones de activación no lineales (o son la identidad). Entonces, f y g son transformaciones lineales representadas por matrices.

- Codificador: $f(\mathbf{x}) = \mathbf{W}_e \mathbf{x}$, donde $\mathbf{W}_e \in \mathbb{R}^{d \times m}$.
- Decodificador: $g(\mathbf{z}) = \mathbf{W}_d \mathbf{z}$, donde $\mathbf{W}_d \in \mathbb{R}^{m \times d}$.
- *Nota:* Asumimos datos centrados (media 0) para simplificar, por lo que omitimos el término de sesgo (*bias*).

2. **Reconstrucción:** La salida del autoencoder es:

$$\hat{\mathbf{x}} = \mathbf{W}_d(\mathbf{W}_e \mathbf{x}) = (\mathbf{W}_d \mathbf{W}_e) \mathbf{x}$$

Definamos $\mathbf{P} = \mathbf{W}_d \mathbf{W}_e$. $\mathbf{P}$ es una matriz de proyección $m \times m$ de rango máximo d .

3. **Minimización del Error:** El problema se reduce a minimizar el error de cuadrados medio sobre el conjunto de datos $\mathbf{X}$:

$$\min_{\mathbf{W}_e, \mathbf{W}_d} \sum \|\mathbf{x}_i - \mathbf{P} \mathbf{x}_i\|^2$$

Esto es geoméricamente equivalente a encontrar el subespacio de dimensión d que minimiza la distancia perpendicular (error de proyección) de todos los puntos de datos al subespacio.

4. **Conexión con PCA (Teorema de Eckart-Young-Mirsky):** Se demuestra en álgebra lineal que el subespacio que minimiza el error de reconstrucción es exactamente el span de los primeros d vectores propios (eigenvectores) de la matriz de covarianza de los datos, correspondientes a los d valores propios más grandes (máxima varianza).

5. **Conclusión:** Para que el autoencoder lineal alcance este óptimo, las columnas de la matriz del codificador $\mathbf{W}_e$ deben corresponder a estos vectores propios principales (o una base ortonormal del mismo subespacio). Por ende, **el autoencoder lineal óptimo recupera los Componentes Principales.**

4.3. Ventajas No Lineales (1.0 punto)

El PCA es limitado porque solo puede capturar relaciones lineales (hiperplanos). Los Autoencoders no lineales (con activaciones como sigmoide, ReLU, Tanh) superan esto:

1. **Aprendizaje de Variedades No Lineales (Manifold Learning):**

- Los datos del mundo real (imágenes, voz) a menudo yacen sobre variedades curvas de baja dimensión incrustadas en el espacio de alta dimensión.
- PCA intenta aplanar esta variedad con un plano, lo cual causa una gran distorsión.
- Un Autoencoder no lineal puede "desenrollar" o doblar el espacio latente para adaptarse a la geometría compleja de los datos, capturando estructuras que el PCA pierde.

2. **Mayor Expresividad en la Representación Latente:**

- La combinación de funciones no lineales permite que el codificador aprenda características jerárquicas y complejas (bordes $\rightarrow$ formas $\rightarrow$ objetos) que no son combinaciones lineales simples de los píxeles de entrada. Esto resulta en una representación $\mathbf{z}$ más densa y semánticamente rica.

Ejemplo Crítico:

- **Conjunto de datos "Swiss Roll":** Una superficie enrollada en 3D. PCA simplemente proyectaría los datos sobre un plano plano, mezclando puntos que están lejos en la superficie enrollada pero cerca en el espacio 3D. Un Autoencoder no lineal es capaz de "desenrollar" el Swiss Roll en un plano recto en 2D, preservando las distancias reales entre los datos a lo largo de la superficie.

Pregunta 5. CNN y Jerarquía Visual (3 puntos)

1. Inspiración Biológica (1 punto)

La arquitectura de las CNNs fue diseñada para emular el sistema visual cortical (descubierto por Hubel y Wiesel), organizado jerárquicamente:

Nivel Biológico (Corteza Visual)	Analogía en CNN	Características Detectadas (Ejemplos)
Células Simples (V1)	Capas Convolucionales Tempranas	Responden a estímulos específicos en una posición exacta. Detectan bordes orientados, líneas rectas y contrastes de color simples.
Células Complejas (V1/V2)	Capas de Pooling	Agregan la salida de células simples. Son más invariantes a la posición exacta. Detectan texturas y patrones repetitivos independientemente de su localización micro.
Células Hipercomplejas (V4/IT)	Capas Profundas / Fully Connected	Combinan características previas para representar formas complejas. Detectan formas globales, partes de objetos (ojos, ruedas) y objetos completos (caras, coches).

2. Independencia Condicional y Peso Compartido (1 punto)

Formalización de Independencia Condicional: En una capa convolucional, la activación $Y_{i,j}$ en una posición espacial (i, j) se calcula únicamente sobre el campo receptivo local (parche) X_{local} de la entrada. Dado el filtro W y la entrada X , la distribución de la activación en una posición (i, j) es independiente de las activaciones en posiciones distantes (k, l) , dado que no comparten entradas ni parámetros estocásticos durante la inferencia determinista.

$$P(Y_{i,j}|X, W) = P(Y_{i,j}|X_{local(i,j)}, W)$$

Relación con Peso Compartido e Invariancia:

- Compartición de Pesos:** El mismo filtro W se aplica (desliza) sobre toda la entrada X . Esto significa que el sistema busca la misma característica (ej. un borde vertical) en todas las coordenadas.
- Invariancia a Traslaciones:** Dado que el filtro se comparte, si una característica aparece en la posición (x, y) o movida a $(x + 10, y + 10)$, el filtro producirá la misma respuesta de activación (fuerte) en ambas posiciones del mapa de características. Esto permite que la red reconozca el objeto sin importar dónde esté ubicado en la imagen.

3. Diagrama Arquitectónico y Dimensiones (1 punto)

A continuación se detallan los cálculos de las dimensiones de los tensores capa por capa:

1. Entrada:

- Dimensiones: $32 \times 32 \times 3$ (Ancho $\times$ Alto $\times$ Canales).

2. Capa Convolucional (Conv):

- Parámetros: 6 filtros de 5×5 , Stride 1, Padding 0.
- Cálculo espacial: $\frac{32-5}{1} + 1 = 28$.
- Profundidad: 6 (número de filtros).
- Dimensiones Salida:** $28 \times 28 \times 6$.

3. Activación (ReLU):

- La función de activación se aplica elemento a elemento y no altera las dimensiones.
- Dimensiones Salida:** $28 \times 28 \times 6$.

4. Max Pooling:

- Parámetros: Filtro 2×2 , Stride 2.
- Cálculo espacial: $\frac{28-2}{2} + 1 = 14$.
- Profundidad: Se mantiene igual (6).
- Dimensiones Salida:** $14 \times 14 \times 6$.

5. Flattening (Aplanado) - *Paso intermedio implícito:*

- Para conectar a la capa totalmente conectada, se debe vectorizar el volumen.
- Total elementos: $14 \times 14 \times 6 = 1176$.
- Vector: 1176×1 .

6. Capa Completamente Conectada (FC):

- Entrada: 1176 neuronas.
- Salida: 120 neuronas (definido por el problema).
- Dimensiones Salida:** 120.

7. Capa de Salida (FC Final):

- Entrada: 120 neuronas.
- Salida: 10 neuronas (número de clases).
- Dimensiones Salida:** 10.